

zomato-anaylsis

December 6, 2025

```
[6]: import pandas as pd #->for data import and anlyize
import numpy as np #-> numerical and array import
import matplotlib.pyplot as plt # graph vidulazation
import seaborn as sns # advance vidulazation
import plotly.express as px # adv of seaborn
import plotly.graph_objects as go # multiple plot
df1 = pd.read_csv("Zomato data (1).csv")
df1
```

```
[6]:
```

	name	online_order	book_table	rate	votes	\
0	Jalsa	Yes	Yes	4.1/5	775	
1	Spice Elephant	Yes	No	4.1/5	787	
2	San Churro Cafe	Yes	No	3.8/5	918	
3	Addhuri Udupi Bhojana	No	No	3.7/5	88	
4	Grand Village	No	No	3.8/5	166	
..	...	...	...	...		
143	Melting Melodies	No	No	3.3/5	0	
144	New Indraprasta	No	No	3.3/5	0	
145	Anna Kuteera	Yes	No	4.0/5	771	
146	Darbar	No	No	3.0/5	98	
147	Vijayalakshmi	Yes	No	3.9/5	47	

	approx_cost(for two people)	listed_in(type)
0	800	Buffet
1	800	Buffet
2	800	Buffet
3	300	Buffet
4	600	Buffet
..	...	...
143	100	Dining
144	150	Dining
145	450	Dining
146	800	Dining
147	200	Dining

[148 rows x 7 columns]

```
[13]: #I) What type of restaurant do the majority of customers order from?
#step 1 : Cleaning (not NAN , all floting/int value in data )
# convert the data type of colume rate
def halderate(value):
    value = str(value).split('/')
    value = value[0]
    return float(value)

df1['rate'] = df1['rate'].apply(halderate)
df1

# to check any null value
df1.info()
#non null means not NAN
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 148 entries, 0 to 147
Data columns (total 7 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   name                                148 non-null    object
1   online_order                        148 non-null    object
2   book_table                          148 non-null    object
3   rate                                148 non-null    float64
4   votes                               148 non-null    int64
5   approx_cost(for two people)         148 non-null    int64
6   listed_in(type)                     148 non-null    object
dtypes: float64(1), int64(2), object(4)
memory usage: 8.2+ KB
```

```
[17]: df1.head()
```

```
[17]:
```

	name	online_order	book_table	rate	votes	\
0	Jalsa	Yes	Yes	4.1	775	
1	Spice Elephant	Yes	No	4.1	787	
2	San Churro Cafe	Yes	No	3.8	918	
3	Addhuri Udupi Bhojana	No	No	3.7	88	
4	Grand Village	No	No	3.8	166	

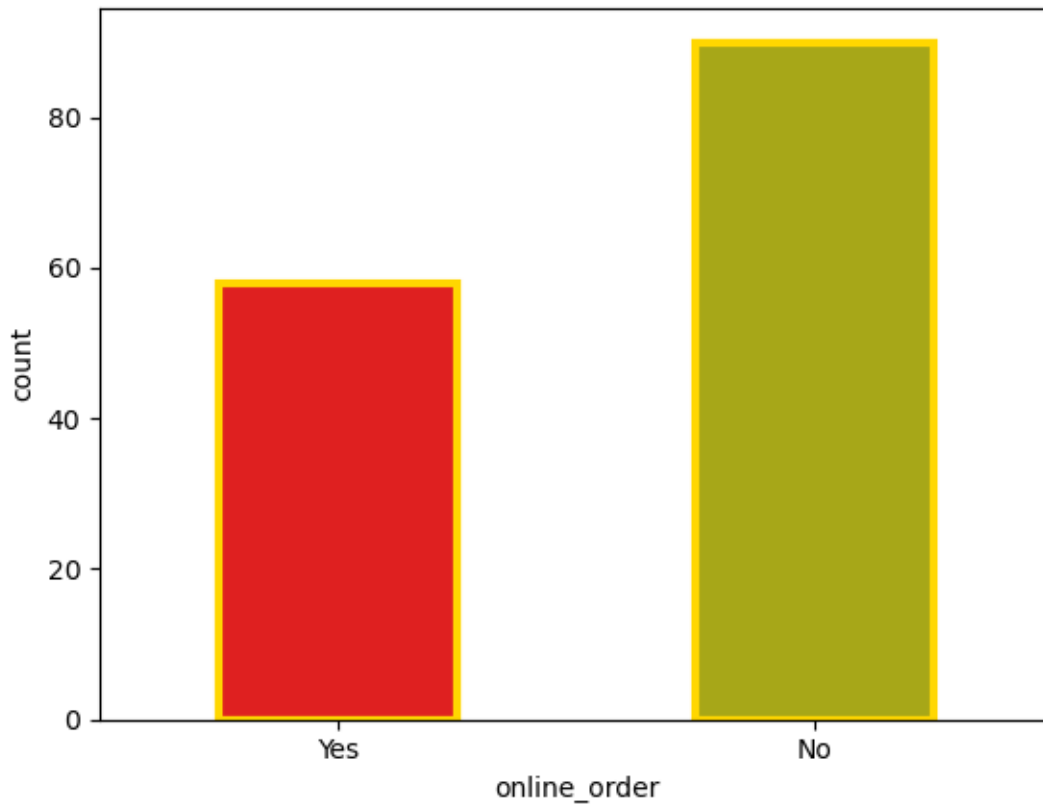
	approx_cost(for two people)	listed_in(type)
0	800	Buffet
1	800	Buffet
2	800	Buffet
3	300	Buffet
4	600	Buffet

```
[19]: df1.describe()
```

```
[19]:
```

	rate	votes	approx_cost(for two people)
count	148.000000	148.000000	148.000000
mean	3.633108	264.810811	418.243243
std	0.402271	653.676951	223.085098
min	2.600000	0.000000	100.000000
25%	3.300000	6.750000	200.000000
50%	3.700000	43.500000	400.000000
75%	3.900000	221.750000	600.000000
max	4.600000	4884.000000	950.000000

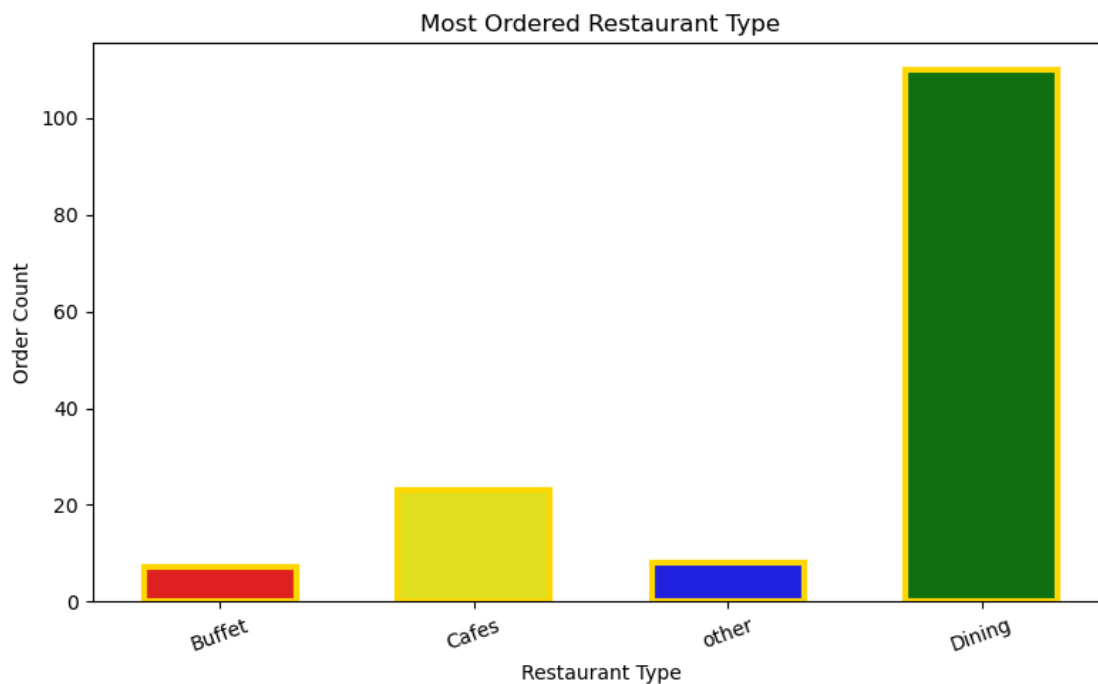
```
[51]: #Q1 make bar graph that how many online vs offline
sns.countplot(
    x='online_order',
    hue='online_order',
    data=df1,
    width=.5,
    palette=['r','y'],
    edgecolor='gold',      # add border color
    linewidth=3           # border thickness
)
plt.show()
```



```
[48]: #Q2 kon se type se order jada hai
# Q2: Which type has the highest number of orders?
plt.figure(figsize=(8,5)) # graph size

sns.countplot(
    x='listed_in(type)',
    hue='listed_in(type)',
    data=df1,
    palette=['red','yellow','blue','green'],
    legend=False,
    width=0.6,
    edgecolor='gold',      # add border color
    linewidth=3           # border thickness
)

plt.title("Most Ordered Restaurant Type") # title
plt.xlabel("Restaurant Type")             # x-axis label
plt.ylabel("Order Count")                 # y-axis label
plt.xticks(rotation=20)                   # rotate for readability
plt.tight_layout()                        # adjust spacing
plt.show()
```



```
[55]: # group each restaurant type and sum their votes
grouped_data = df1.groupby('listed_in(type)')['votes'].sum()
```

```

# convert the grouped result into a DataFrame
result = pd.DataFrame({'votes': grouped_data})

# plot the total votes received by each restaurant type
plt.figure(figsize=(7,4))
plt.plot(result.index, result['votes'], c="green", marker="o") # corrected ↴
    ↪marker
plt.xticks(rotation=20) # rotate x labels
plt.xlabel("Restaurant Type")
plt.ylabel("Total Votes")
plt.title("Total Votes Received by Each Restaurant Type")
plt.tight_layout()
plt.show()

# Example (easy to understand)

# Imagine your data:

# listed_in(type)    votes
# Dessert            100
# Buffet             50
# Dessert             30
# Cafe               20
# Buffet             40

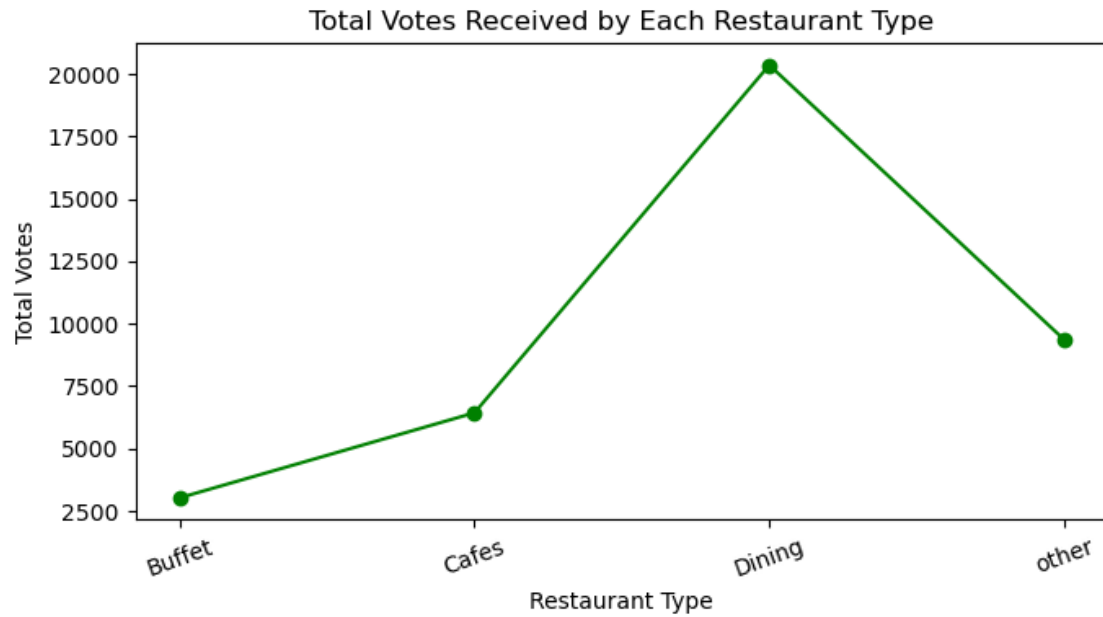
# If you use:

# df1.groupby('listed_in(type)')['votes'].sum()

# It becomes:

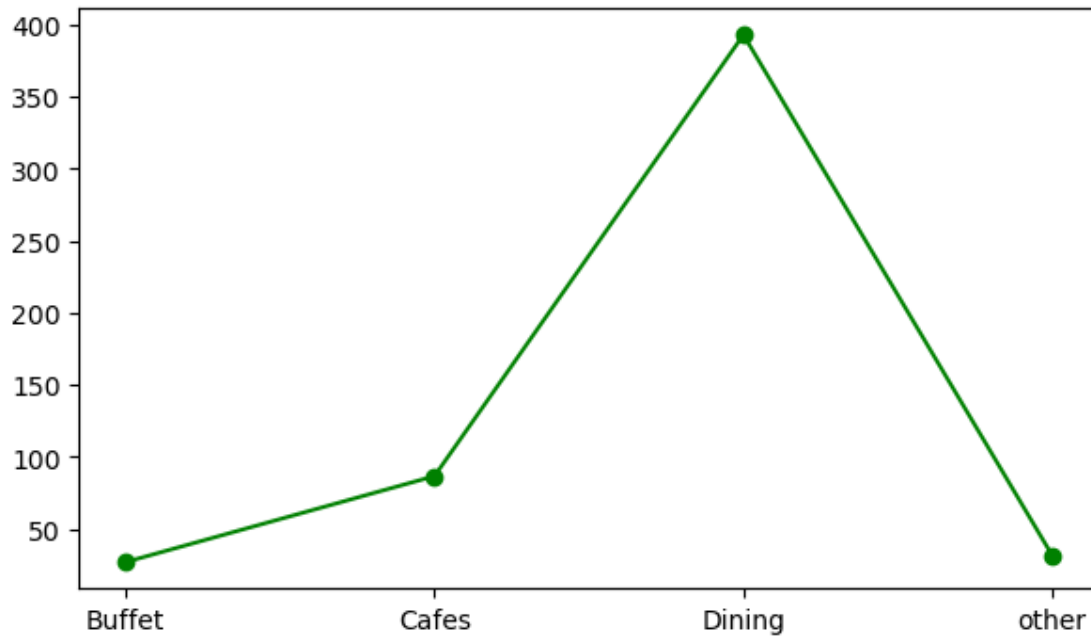
# listed_in(type)    total votes
# Buffet             90
# Cafe               20
# Dessert            130

```



```
[61]: # Q3
groupdata=df1.groupby('listed_in(type)')['rate'].sum()
result = pd.DataFrame({'rate': groupdata})
plt.figure(figsize=(7,4))
plt.plot(result.index, result['rate'], c="green", marker="o") # corrected ↵
↵marker
```

```
[61]: [<matplotlib.lines.Line2D at 0x207e4e4c7d0>]
```



```
[64]: # group each restaurant type and calculate average rating
grouped = df1.groupby('listed_in(type)')['rate'].mean()

# convert to DataFrame
result = grouped.reset_index()

# plot rate vs restaurant type
plt.figure(figsize=(6,4))

plt.bar(result['listed_in(type)'],          # x-axis → restaurant types
        result['rate'],                    # y-axis → average rate
        color='green',                     # bar color
        edgecolor='black')                 # border for bars

plt.xlabel("Restaurant Type")              # x-axis label
plt.ylabel("Average Rating")               # y-axis label
plt.title("Restaurant Type vs Rating")     # title
plt.xticks(rotation=20)                    # slight rotation for readability
plt.tight_layout()                         # avoid cut-off
plt.show()

# Example data (very simple)
# listed_in(type)  rate
# Buffet          4.1
# Buffet          3.8
```

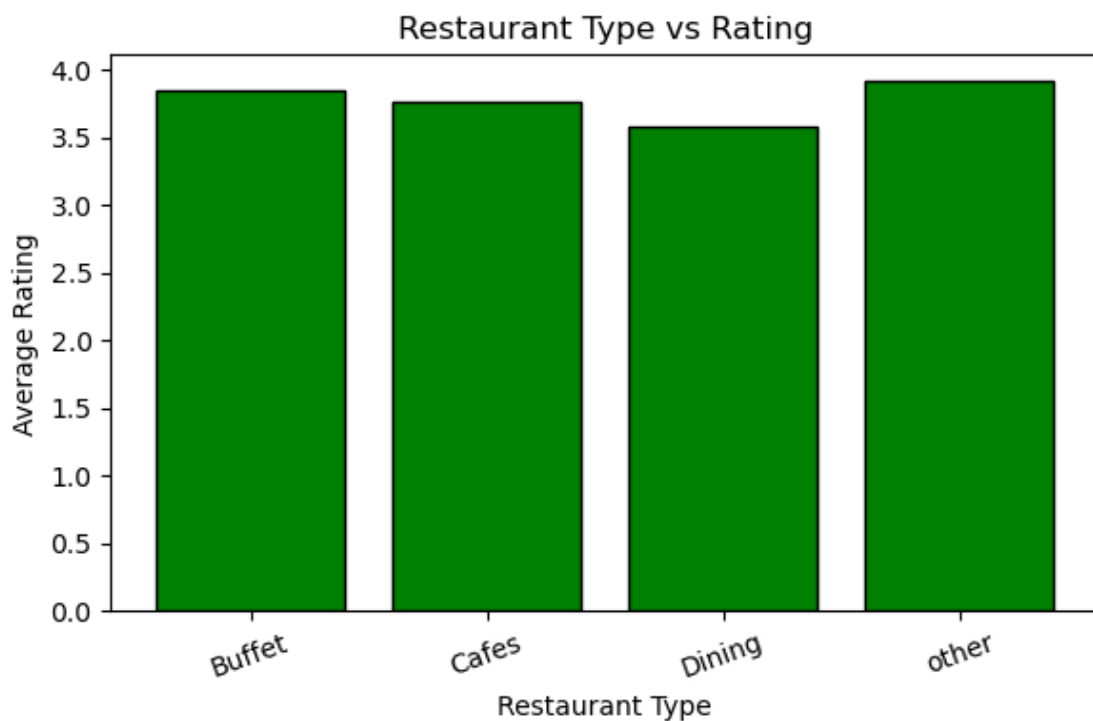
```

# Cafe          4.2
# Cafe          3.9
# Dessert       4.5
# Quick Bites   3.7

# Group same restaurant types and calculate average rate
# Buffet → (4.1 + 3.8)/2 = 3.95
# Cafe → (4.2 + 3.9)/2 = 4.05
# Dessert → 4.5
# Quick Bites → 3.7

# Resulting average rates per type:
# listed_in(type)  Average Rate
# Buffet          3.95
# Cafe            4.05
# Dessert         4.5
# Quick Bites     3.7

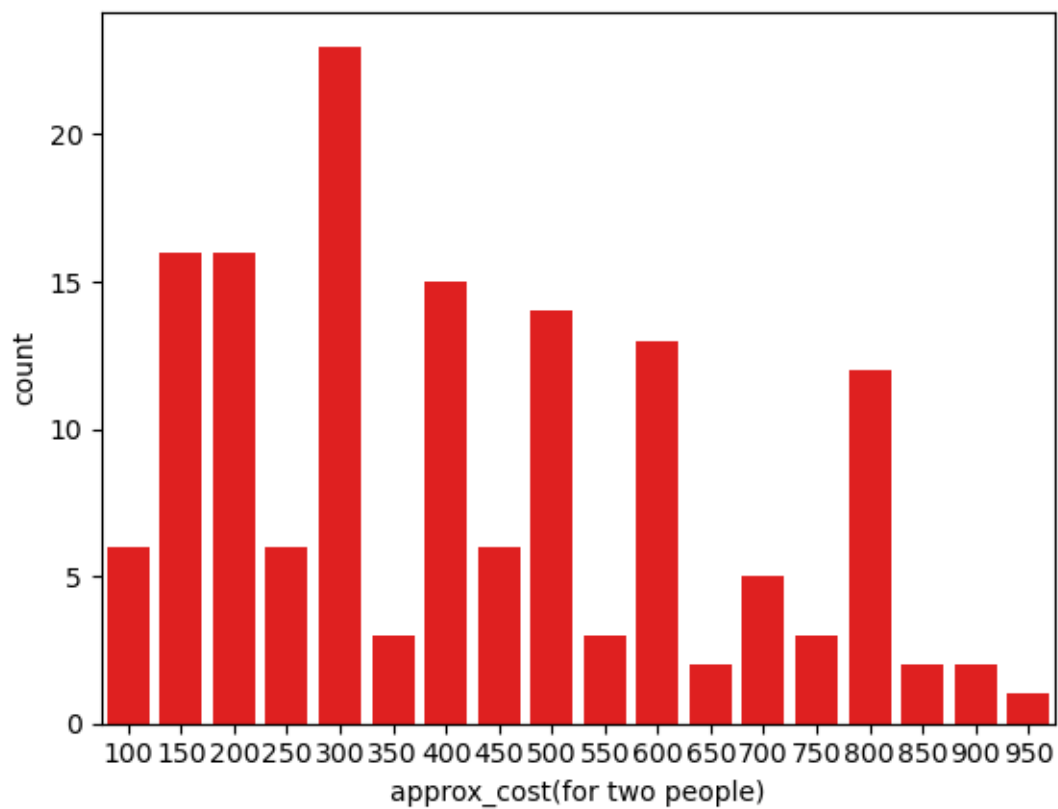
```



```

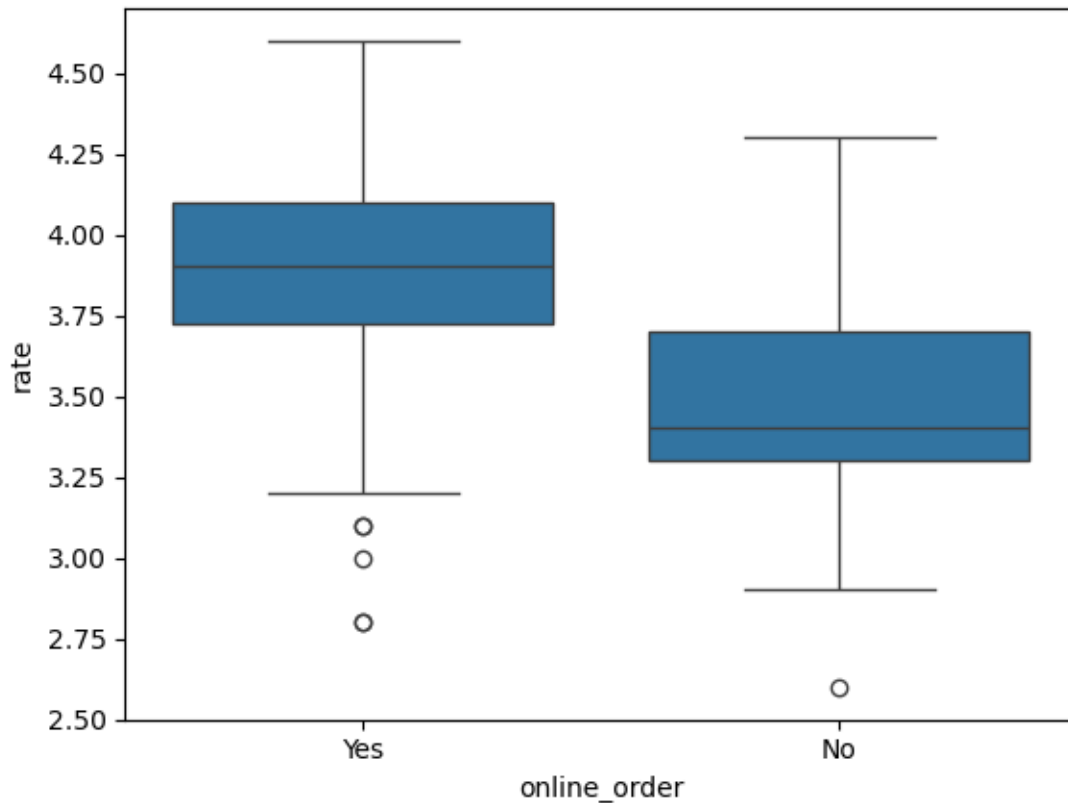
[68]: #4) Zomato has observed that most couples order most of their food online. What
      ↪ is their
      #average spending on each order?
      sns.countplot(x=df1['approx_cost(for two people)'], color="r")
      plt.show()

```

```
[69]: # box graph
sns.boxplot(x="online_order",y="rate",data=df1)
```

```
[69]: <Axes: xlabel='online_order', ylabel='rate'>
```

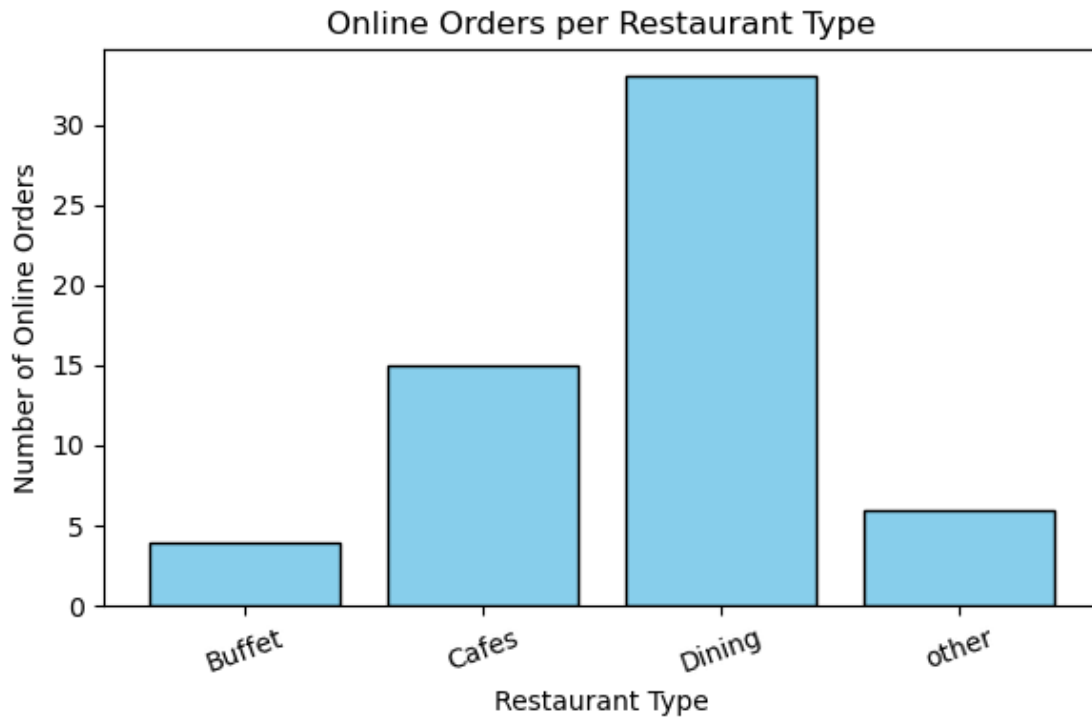


```
[71]: # Convert Yes/No to numeric
df1['online_order_num'] = df1['online_order'].map({'Yes': 1, 'No': 0})

# Group by restaurant type and sum online orders
grouped_data = df1.groupby('listed_in(type)')['online_order_num'].sum()

# Convert to DataFrame
result = pd.DataFrame({'online_order_count': grouped_data})

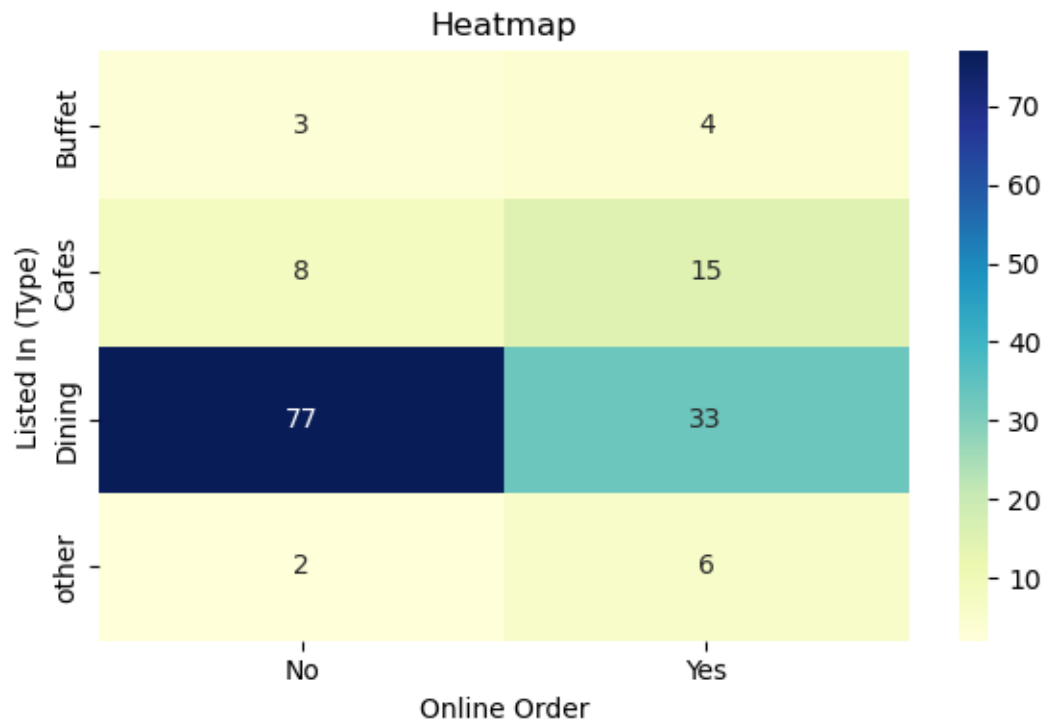
# Plot online orders per restaurant type
plt.figure(figsize=(6,4))
plt.bar(result.index, result['online_order_count'], color='skyblue',
        edgecolor='black')
plt.xlabel("Restaurant Type")
plt.ylabel("Number of Online Orders")
plt.title("Online Orders per Restaurant Type")
plt.xticks(rotation=20)
plt.tight_layout()
plt.show()
```



```
[72]: import seaborn as sns
import matplotlib.pyplot as plt

# create a pivot table with restaurant type vs online order counts
pivot_table = df1.pivot_table(
    index='listed_in(type)',      # rows → restaurant types
    columns='online_order',      # columns → Yes/No
    aggfunc='size',              # count number of entries
    fill_value=0,                # replace missing values with 0
)

# plot heatmap
plt.figure(figsize=(6,4))
sns.heatmap(pivot_table, annot=True, cmap="YlGnBu", fmt='d') # annot=True to show numbers
plt.title("Heatmap")
plt.xlabel("Online Order")
plt.ylabel("Listed In (Type)")
plt.tight_layout()
plt.show()
```



[]: